```
# Import required libraries

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

from sklearn.linear_model import LogisticRegression

from scipy.stats import chi2_contingency

print("All libraries imported successfully!")
```

```
All libraries imported successfully!
```

```
# Upload the dataset

uploaded = files.upload()

filename = list(uploaded.keys())[0]

print("Uploaded file:", filename)

# Read CSV dataset
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
```

Choose Files Bank Custo…ediction.csv

Bank Customer Churn Prediction.csv(text/csv) - 561600 bytes, last modified: 9/22/2026 - 100% done

```
Saving Bank Customer Churn Prediction.csv to Bank Customer Churn Prediction.csv
Uploaded file: Bank Customer Churn Prediction.csv
Dataset loaded successfully!
```

```
# Display first 10 rows

df.head(10)
```

	customer_id	credit_score	country	gender	age	tenure	balance	products_number	credit_card	active_member	estimated_sal
0	15634602	619	France	Female	42	2	0.00	1	1	1	10134
1	15647311	608	Spain	Female	41	1	83807.86	1	0	1	11254
2	15619304	502	France	Female	42	8	159660.80	3	1	0	11393
3	15701354	699	France	Female	39	1	0.00	2	0	0	9382
4	15737888	850	Spain	Female	43	2	125510.82	1	1	1	7908
5	15574012	645	Spain	Male	44	8	113755.78	2	1	0	14975
6	15592531	822	France	Male	50	7	0.00	2	1	1	1006
7	15656148	376	Germany	Female	29	4	115046.74	4	1	0	11934
8	15792365	501	France	Male	44	4	142051.07	2	0	1	7494
9	15592389	684	France	Male	27	2	134603.88	1	1	1	7172

Next steps: Generate code with df New interactive sheet

```
# Display dataset dimensions

print("Number of Rows:", df.shape[0])
print("Number of Columns:", df.shape[1])
```

```
Number of Rows: 10000
Number of Columns: 12
```

```
# Display all column names

print("Columns in the dataset:")

for i, col in enumerate(df.columns, 1):
    print(i, ":", col)
```

```
Columns in the dataset:
1 : customer_id
2 : credit_score
3 : country
4 : gender
5 : age
6 : tenure
7 : balance
8 : products_number
9 : credit_card
10 : active_member
11 : estimated_salary
12 : churn
```

```
# Display dataset information

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customer_id       10000 non-null  int64
 1   credit_score      10000 non-null  int64
 2   country           10000 non-null  object
 3   gender            10000 non-null  object
 4   age               10000 non-null  int64
 5   tenure            10000 non-null  int64
 6   balance           10000 non-null  float64
 7   products_number   10000 non-null  int64
 8   credit_card       10000 non-null  int64
 9   active_member     10000 non-null  int64
 10  estimated_salary  10000 non-null  float64
 11  churn             10000 non-null  int64
dtypes: float64(2), int64(8), object(2)
memory usage: 937.6+ KB
```

```
# Descriptive statistics

df.describe(include='all').T
```

	count	unique	top	freq	mean	std	min	25%	50%	75%	
customer_id	10000.0	NaN	NaN	NaN	15690940.5694	71936.186123	15565701.0	15628528.25	15690738.0	15753233.75	1581
credit_score	10000.0	NaN	NaN	NaN	650.5288	96.653299	350.0	584.0	652.0	718.0	
country	10000	3	France	5014	NaN	NaN	NaN	NaN	NaN	NaN	
gender	10000	2	Male	5457	NaN	NaN	NaN	NaN	NaN	NaN	
age	10000.0	NaN	NaN	NaN	38.9218	10.487806	18.0	32.0	37.0	44.0	
tenure	10000.0	NaN	NaN	NaN	5.0128	2.892174	0.0	3.0	5.0	7.0	
balance	10000.0	NaN	NaN	NaN	76485.889288	62397.405202	0.0	0.0	97198.54	127644.24	250
products_number	10000.0	NaN	NaN	NaN	1.5302	0.581654	1.0	1.0	1.0	2.0	
credit_card	10000.0	NaN	NaN	NaN	0.7055	0.45584	0.0	0.0	1.0	1.0	
active_member	10000.0	NaN	NaN	NaN	0.5151	0.499797	0.0	0.0	1.0	1.0	
estimated_salary	10000.0	NaN	NaN	NaN	100090.239881	57510.492818	11.58	51002.11	100193.915	149388.2475	199
churn	10000.0	NaN	NaN	NaN	0.2037	0.402769	0.0	0.0	0.0	0.0	

```
# Check missing values

missing_values = df.isnull().sum()

print("Missing values in each column:")
print(missing_values)

print("\nTotal missing values:",
      df.isnull().sum().sum())
```

```
Missing values in each column:
customer_id         0
credit_score        0
country             0
gender              0
age                 0
tenure              0
balance             0
products_number     0
credit_card         0
active_member       0
estimated_salary    0
churn               0
dtype: int64

Total missing values: 0
```

```
# Handle missing values

for col in df.columns:

    if df[col].isnull().sum() > 0:

        if df[col].dtype == 'object':
            df[col] = df[col].fillna(df[col].mode()[0])

        else:
            df[col] = df[col].fillna(df[col].median())

print("Missing values handled successfully!")

print("\nRemaining missing values:")
print(df.isnull().sum())
```

```
Missing values handled successfully!

Remaining missing values:
customer_id         0
credit_score        0
country             0
gender              0
age                 0
tenure              0
balance             0
products_number     0
```

```
credit_card         0
active_member       0
estimated_salary    0
churn               0
dtype: int64
```

```
# Check duplicate records

duplicates = df.duplicated().sum()

print("Number of duplicate records:", duplicates)

# Remove duplicates
df = df.drop_duplicates()

print("Dataset shape after removing duplicates:", df.shape)
```

```
Number of duplicate records: 0
Dataset shape after removing duplicates: (10000, 12)
```

```
# Identify target variable

target_candidates = ['Exited', 'Churn', 'churn', 'Attrition', 'attrition']

target_col = None

for col in target_candidates:
    if col in df.columns:
        target_col = col
        break

print("Target variable:", target_col)
```

```
Target variable: churn
```

```
# Display target distribution

if target_col:

    print(df[target_col].value_counts())

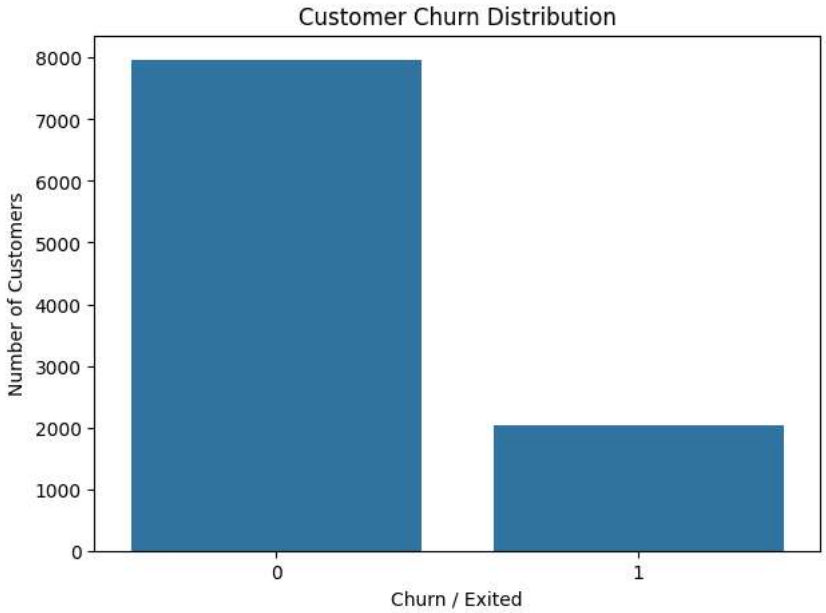
    plt.figure(figsize=(7,5))

    sns.countplot(x=df[target_col])

    plt.title("Customer Churn Distribution")
    plt.xlabel("Churn / Exited")
    plt.ylabel("Number of Customers")

    plt.show()
```

```
churn
0    7963
1    2037
Name: count, dtype: int64
```




```
# Select numerical columns

numeric_columns = df.select_dtypes(include=np.number).columns

print("Numerical columns:")
print(list(numeric_columns))
```

```
Numerical columns:
['customer_id', 'credit_score', 'age', 'tenure', 'balance', 'products_number', 'credit_card', 'active_member', 'estimated_salary
```

```
# Correlation analysis

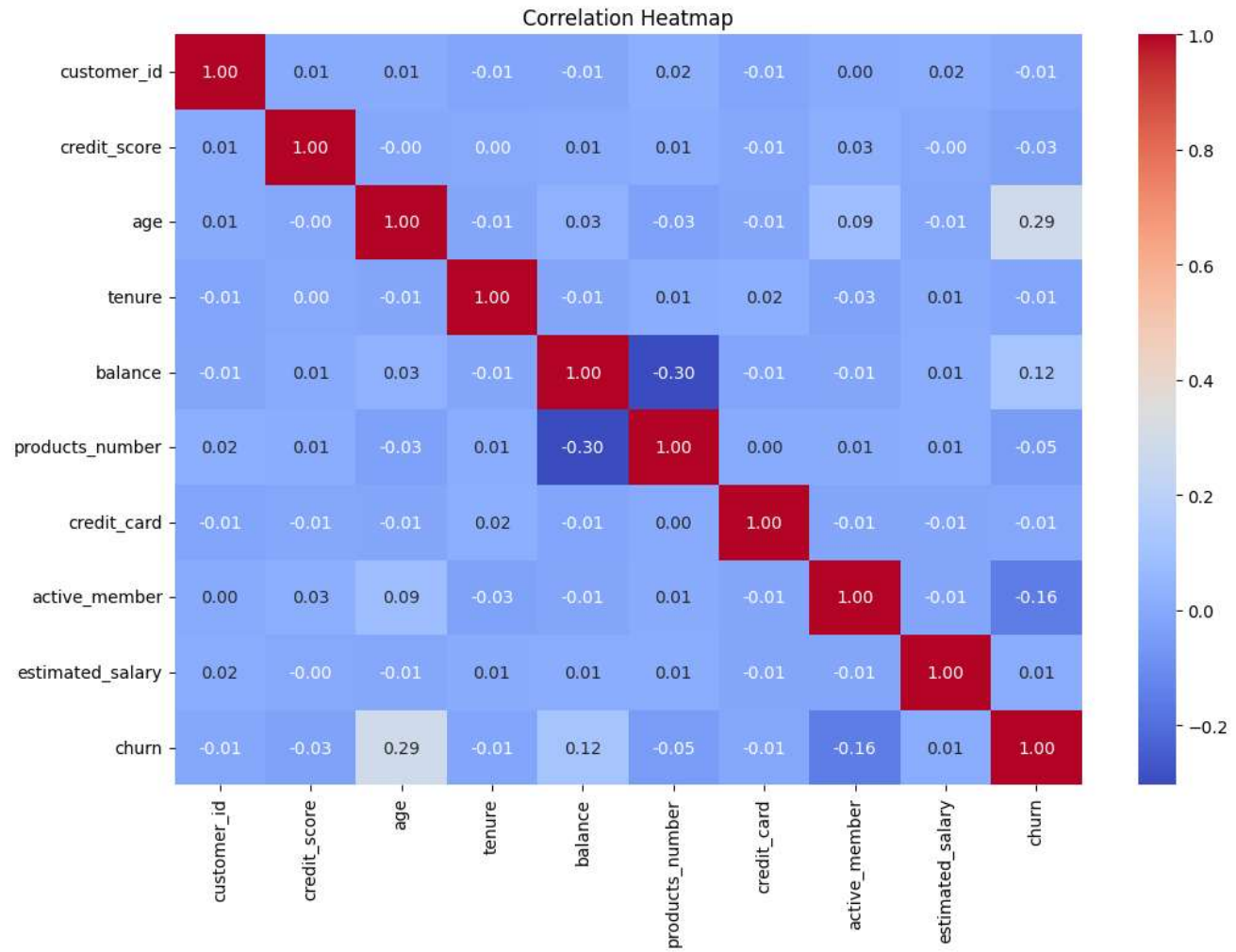
plt.figure(figsize=(12,8))

correlation = df[numeric_columns].corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Correlation Heatmap")

plt.show()
```




```
# Find Age column

age_col = None

for col in df.columns:
    if col.lower() == 'age':
        age_col = col
        break

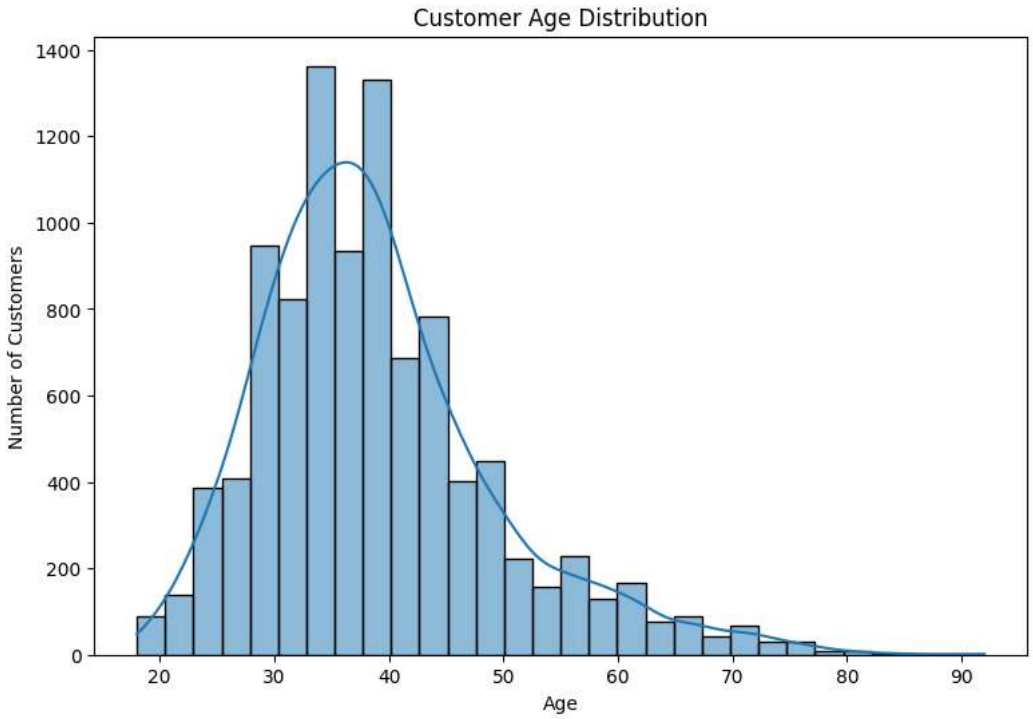
if age_col:

    plt.figure(figsize=(9,6))

    sns.histplot(
        df[age_col],
        bins=30,
        kde=True
    )

    plt.title("Customer Age Distribution")
    plt.xlabel("Age")
    plt.ylabel("Number of Customers")

    plt.show()
```




```
# Find credit score column

credit_col = None

for col in df.columns:

    if col.lower().replace(" ", "") in [
        "creditscore",
        "credit_score"
    ]:
        credit_col = col
        break

print("Credit Score column:", credit_col)

if credit_col:

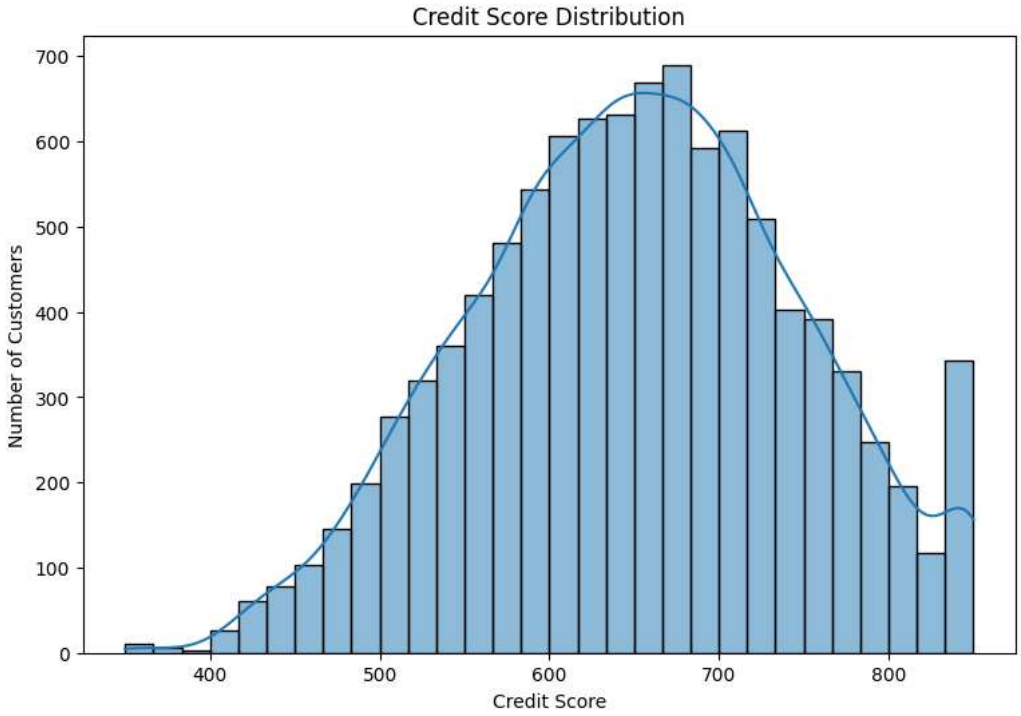
    plt.figure(figsize=(9,6))

    sns.histplot(
        df[credit_col],
        bins=30,
        kde=True
    )

    plt.title("Credit Score Distribution")
    plt.xlabel("Credit Score")
    plt.ylabel("Number of Customers")

    plt.show()
```

```
Credit Score column: credit_score
```




```
# Find Geography column

geo_col = None

for col in df.columns:
    if col.lower() == 'geography':
        geo_col = col
        break

print("Geography column:", geo_col)

if geo_col and target_col:

    plt.figure(figsize=(9,6))

    sns.countplot(
        data=df,
        x=geo_col,
        hue=target_col
    )

    plt.title("Customer Churn by Geography")

    plt.xlabel("Geography")
    plt.ylabel("Number of Customers")

    plt.show()
```

```
Geography column: None
```

```
# Find gender column

gender_col = None

for col in df.columns:
    if col.lower() == 'gender':
        gender_col = col
        break

print("Gender column:", gender_col)

if gender_col and target_col:

    plt.figure(figsize=(8,6))
```

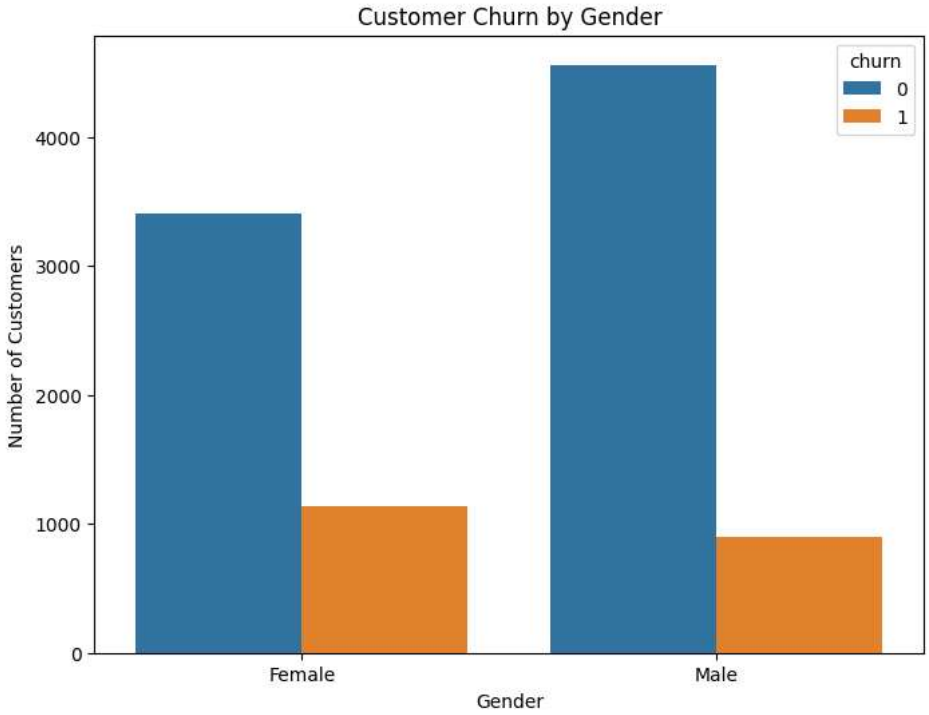
```
    sns.countplot(
        data=df,
        x=gender_col,
        hue=target_col
    )

    plt.title("Customer Churn by Gender")

    plt.xlabel("Gender")
    plt.ylabel("Number of Customers")

    plt.show()
```

```
Gender column: gender
```



```
# Find number of products column

products_col = None

for col in df.columns:

    name = col.lower().replace(" ", "")

    if name in [
        "numofproducts",
        "numberofproducts"
    ]:
        products_col = col
        break

print("Products column:", products_col)

if products_col and target_col:

    plt.figure(figsize=(9,6))

    sns.countplot(
        data=df,
        x=products_col,
        hue=target_col
    )

    plt.title("Customer Churn by Number of Products")

    plt.xlabel("Number of Products")
    plt.ylabel("Number of Customers")

    plt.show()
```

```
Products column: None
```

```
# Display correlation with target variable

if target_col:

    correlation_with_target = (
        df[numeric_columns]
        .corr()[target_col]
        .sort_values(ascending=False)
    )

    print("Correlation with target variable:")
    print(correlation_with_target)
```

```
Correlation with target variable:
churn               1.000000
age                 0.285323
balance             0.118533
estimated_salary    0.012097
customer_id        -0.006248
credit_card        -0.007138
tenure             -0.014001
credit_score       -0.027094
products_number    -0.047820
active_member      -0.156128
Name: churn, dtype: float64
```

```
# Chi-square test between Geography and Churn

if geo_col and target_col:

    contingency_table = pd.crosstab(
        df[geo_col],
        df[target_col]
    )

    print("Contingency Table:")
    display(contingency_table)

    chi2, p, dof, expected = chi2_contingency(
        contingency_table
    )
```

```
    print("\nChi-Square Statistic:", chi2)
    print("P-value:", p)
    print("Degrees of Freedom:", dof)

    alpha = 0.05

    if p < alpha:
        print("\nResult: There is a statistically significant relationship.")
    else:
        print("\nResult: No statistically significant relationship was detected.")
```

```
# Make a copy of the dataset

ml_df = df.copy()

# Remove commonly unnecessary identifier columns

columns_to_remove = []

for col in ml_df.columns:

    name = col.lower().replace(" ", "")

    if name in [
        "rownumber",
        "customerid",
        "customer_id",
        "surname"
    ]:
        columns_to_remove.append(col)

ml_df = ml_df.drop(
    columns=columns_to_remove,
    errors='ignore'
)

print("Removed columns:", columns_to_remove)

print("\nRemaining columns:")
print(ml_df.columns.tolist())
```

```
Removed columns: ['customer_id']

Remaining columns:
['credit_score', 'country', 'gender', 'age', 'tenure', 'balance', 'products_number', 'credit_card', 'active_member', 'estimated_
```

```
# Convert categorical variables into numerical variables

categorical_columns = ml_df.select_dtypes(
    include=['object']
).columns.tolist()

print("Categorical columns:")
print(categorical_columns)

ml_df = pd.get_dummies(
    ml_df,
    columns=categorical_columns,
    drop_first=True
)

print("\nEncoded dataset shape:", ml_df.shape)
```

```
Categorical columns:
['country', 'gender']

Encoded dataset shape: (10000, 12)
```

```
# Separate independent variables and target variable

X = ml_df.drop(columns=[target_col])
y = ml_df[target_col]
```

```
print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Feature shape: (10000, 11)
Target shape: (10000,)
```

```
# Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 8000
Testing samples: 2000
```

```
# Standardize numerical features

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling completed!")
```

```
Feature scaling completed!
```

```
# Create Logistic Regression model

model = LogisticRegression(
    max_iter=1000,
    random_state=42
)

# Train model
model.fit(
    X_train_scaled,
    y_train
)

print("Logistic Regression model trained successfully!")
```

```
Logistic Regression model trained successfully!
```

```
# Make predictions

y_pred = model.predict(X_test_scaled)

print("Predictions generated successfully!")
```

```
Predictions generated successfully!
```

```
# Calculate classification metrics

accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
```

```
    y_test,
    y_pred,
    zero_division=0
)

print("=" * 50)
print("             MODEL PERFORMANCE")
print("=" * 50)

print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1-Score  : {f1:.4f}")

print("=" * 50)
```

```
==================================================
             MODEL PERFORMANCE
==================================================
Accuracy  : 0.8080
Precision : 0.5891
Recall    : 0.1867
F1-Score  : 0.2836
==================================================
```

```
# Detailed classification report

print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)
```

```
              precision    recall  f1-score   support

           0       0.82      0.97      0.89      1593
           1       0.59      0.19      0.28       407

    accuracy                           0.81      2000
   macro avg       0.71      0.58      0.59      2000
weighted avg       0.78      0.81      0.77      2000
```

```
# Confusion Matrix

cm = confusion_matrix(
    y_test,
    y_pred
)

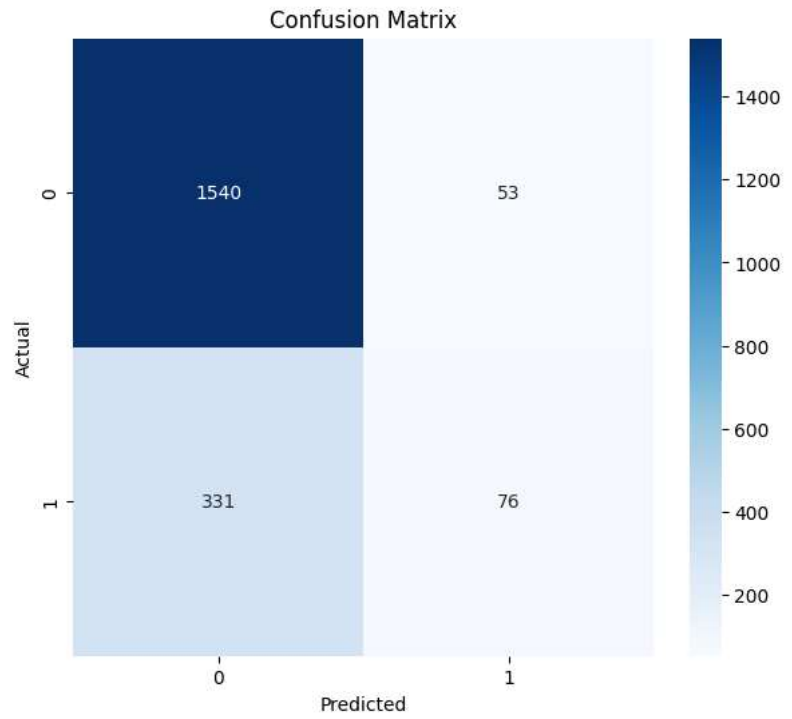
plt.figure(figsize=(7,6))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title("Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```




```
# Feature coefficients

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_[0]
})

feature_importance['Absolute_Coefficient'] = (
    feature_importance['Coefficient'].abs()
)

feature_importance = feature_importance.sort_values(
    by='Absolute_Coefficient',
    ascending=False
)

display(
    feature_importance.head(15)
)
```

	Feature	Coefficient	Absolute_Coefficient
1	age	0.738847	0.738847
6	active_member	-0.515485	0.515485
8	country_Germany	0.356679	0.356679
10	gender_Male	-0.260851	0.260851
3	balance	0.160622	0.160622
0	credit_score	-0.085986	0.085986
4	products_number	-0.070292	0.070292
7	estimated_salary	0.047725	0.047725
5	credit_card	-0.032208	0.032208
2	tenure	-0.020071	0.020071
9	country_Spain	0.018907	0.018907

```
# Plot most important features

top_features = feature_importance.head(10)

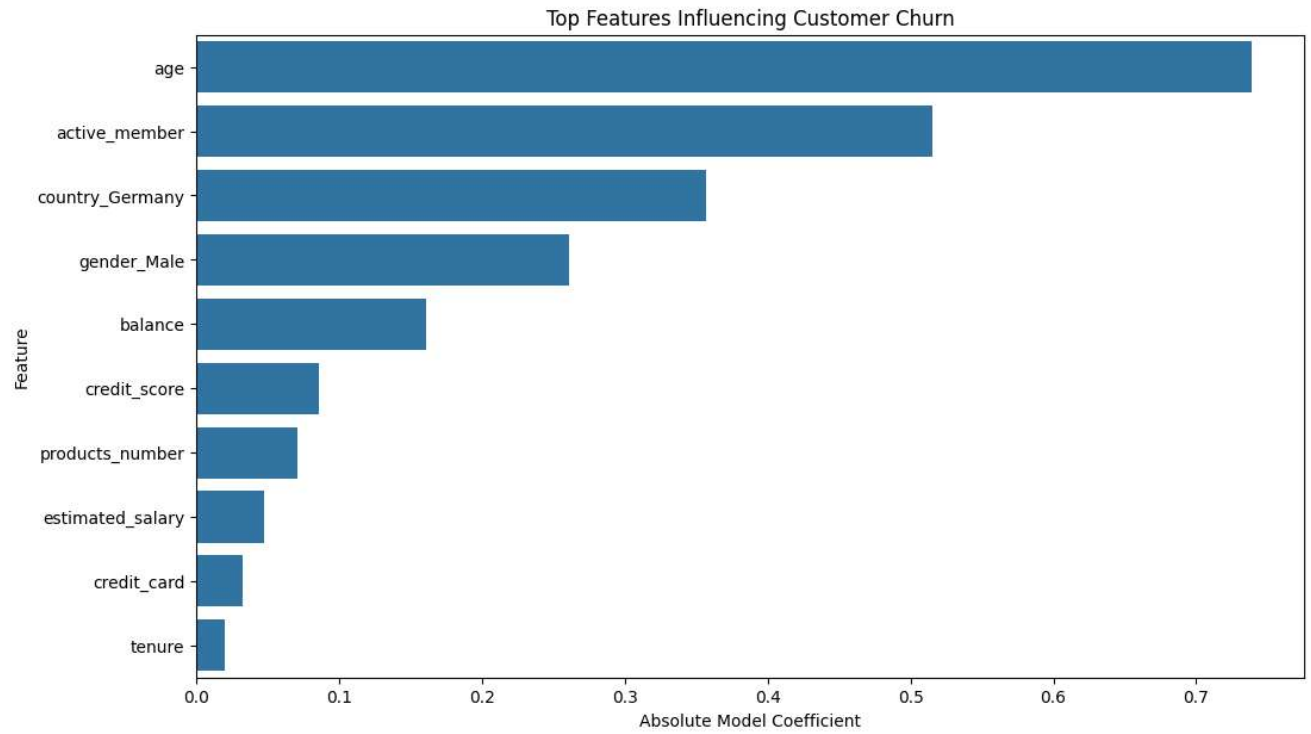
plt.figure(figsize=(12,7))
```

```
sns.barplot(
    data=top_features,
    x='Absolute_Coefficient',
    y='Feature'
)

plt.title("Top Features Influencing Customer Churn")

plt.xlabel("Absolute Model Coefficient")
plt.ylabel("Feature")

plt.show()
```



THEORY QUESTIONS AND ANSWERS

Q1. What are the major stages involved in an end-to-end data analytics

project?

Answer: The major stages of an end-to-end data analytics project are:

1. Problem definition
2. Data collection
3. Data preprocessing
4. Exploratory Data Analysis (EDA)
5. Statistical analysis
6. Data visualization
7. Feature engineering
8. Machine learning model development
9. Model evaluation
10. Interpretation of results
11. Business insights and recommendations
12. Presentation and deployment

These stages transform raw data into useful information and actionable business insights.

Q2. Why is data preprocessing considered the foundation of successful

analytics and machine learning?

Answer: Data preprocessing is considered the foundation because real-world datasets often contain missing values, duplicate records, outliers, inconsistent formats, and categorical variables.

Preprocessing improves data quality and prepares the dataset for analysis and machine learning. Common preprocessing activities include:

• Handling missing values • Removing duplicate records • Detecting and handling outliers • Encoding categorical variables • Scaling numerical variables • Removing unnecessary features

Good preprocessing helps improve the reliability and performance of analytical and machine learning models.

Q3. How does Exploratory Data Analysis (EDA) contribute to model development?

Answer: Exploratory Data Analysis helps understand the structure and characteristics of a dataset before building a machine learning model.

EDA helps identify:

• Important variables • Data distributions • Missing values • Outliers • Relationships between variables • Correlations • Patterns and trends • Potential data quality problems

The findings from EDA can guide feature selection, preprocessing, feature engineering, and algorithm selection. Therefore, EDA helps build more appropriate and reliable predictive models.

Q4. What factors should be considered while selecting a machine learning

algorithm for a business problem?

Answer: The following factors should be considered:

1. Type of problem
    - Classification
    - Regression
    - Clustering
2. Type and size of dataset
3. Number and nature of features
4. Relationship between input and target variables
5. Model interpretability
6. Computational requirements
7. Training time
8. Prediction accuracy
9. Business requirements
10. Available evaluation metrics

For example, Logistic Regression can be suitable for binary classification, while Linear Regression can be used for continuous-value prediction.

Q5. Why is model evaluation essential before deploying a predictive model?

Answer: Model evaluation is essential because a trained model may not always perform well on unseen data.